

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

ΜΕΡΟΣ Β' ΕΡΓΑΣΤΗΡΙΟ

ΑΝΑΠΤΥΞΗ ΛΟΓΙΣΜΙΚΟΥ ΣΕ C# ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ
ΜΕ ΧΡΗΣΗ UNIT TESTS

ΣΤΟΙΧΕΙΑ ΜΕΛΩΝ ΟΜΑΔΑΣ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΑΘΑΝΑΣΙΟΥ ΒΑΣΙΛΕΙΟΣ ΕΥΑΓΓΕΛΟΣ

ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390005

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΘΕΟΧΑΡΗΣ ΓΕΩΡΓΙΟΣ

ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390283

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΟΜΑΡ ΑΛΧΑΖ ΟΜΑΡ

ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390010

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

ΠΕΡΙΕΧΟΜΕΝΑ

A. Ανάπτυξη Βιβλιοθήκης Συναρτήσεων (DLL) για την Διαχείριση Προσωπικού	4
1. bool ValidName(string Name)	4
1.1. Λειτουργία	4
1.2. Υλοποίηση	4
2. bool ValidPassword(string Password).....	4
2.1. Λειτουργία	4
2.2. Υλοποίηση	5
3. EncryptPassword(string Password, ref string EncryptedPW)	5
3.1. Λειτουργία	5
3.2. Υλοποίηση	5
4. CheckPhone(string Phone, ref int TypePhone, ref string InfoPhone)	6
4.1. Λειτουργία	6
4.2. Υλοποίηση	6
5. InfoEmployee(Employee EmpIX, ref int Age, ref int YearsOfExperience)	6
5.1. Λειτουργία	6
5.2. Υλοποίηση	6
6. int LiveInAthens(Employee Empls)	7
6.1. Λειτουργία	7
6.2. Υλοποίηση	7
B. Δημιουργία Περιπτώσεων Ελέγχου (TestCases)	7
1. ValidName Test Case	7
2. ValidPassword Test Case	9
3. EncryptPassword Test Case	10
4. CheckPhone Test Case	11
5. InfoEmployee Test Case	13
6. LiveInAthens Test Case	15
Γ. Ανάπτυξη Ελέγχων Μονάδων (UnitTests)	17
1. ValidName Unit Test.....	17
2. ValidPassword Unit Test	19
3. EncryptPassword Unit Test	21
4. CheckPhone Unit Test	22
5. InfoEmployee Unit Test	25
6. LiveInAthens Unit Test.....	27
Δ. Αναφορές Ελέγχου	31
1. Σφάλμα Ονοματεπωνύμου Υπαλλήλου	31
1.1. Αναγνωριστικό Σφάλματος	31

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

1.2. Όνομα μεθόδους ελέγχου που απέτυχε	31
1.3. Περιγραφή ελέγχου που πραγματοποιήθηκε.....	31
1.4. Αναφορά Σφάλματος που παρουσιάστηκε.....	31
1.5. Στιγμιότυπο οθόνης του Test Explorer.....	32
2. Σφάλμα Κωδικού Πρόσβασης	32
2.1. Αναγνωριστικό Σφάλματος	32
2.2. Όνομα μεθόδους ελέγχου που απέτυχε	32
2.3. Περιγραφή ελέγχου που πραγματοποιήθηκε.....	32
2.4. Αναφορά Σφάλματος που παρουσιάστηκε.....	32
2.5. Στιγμιότυπο οθόνης του Test Explorer.....	33
3. Σφάλμα Τηλεφώνου	33
3.1. Αναγνωριστικό Σφάλματος	33
3.2. Όνομα μεθόδους ελέγχου που απέτυχε	33
3.3. Περιγραφή ελέγχου που πραγματοποιήθηκε.....	33
3.4. Αναφορά Σφάλματος που παρουσιάστηκε.....	34
3.5. Στιγμιότυπο οθόνης του Test Explorer.....	34
4. Σφάλμα Ηλικίας Υπαλλήλου.....	34
4.1. Αναγνωριστικό Σφάλματος	34
4.2. Όνομα μεθόδους ελέγχου που απέτυχε	35
4.3. Περιγραφή ελέγχου που πραγματοποιήθηκε.....	35
4.4. Αναφορά Σφάλματος που παρουσιάστηκε.....	35
4.5. Στιγμιότυπο οθόνης του Test Explorer.....	36
5. Σφάλμα στα Χρόνια Προϋπηρεσίας ενός Υπαλλήλου.....	36
5.1. Αναγνωριστικό Σφάλματος	36
5.2. Όνομα μεθόδους ελέγχου που απέτυχε	36
5.3. Περιγραφή ελέγχου που πραγματοποιήθηκε.....	36
5.4. Αναφορά Σφάλματος που παρουσιάστηκε.....	36
5.5. Στιγμιότυπο οθόνης του Test Explorer.....	37

Α. Ανάπτυξη Βιβλιοθήκης Συναρτήσεων (DLL) για την Διαχείριση Προσωπικού

1. bool ValidName(string Name)

1.1. Λειτουργία

Η συνάρτηση αυτή δέχεται ως παράμετρο ένα κείμενο και επιστρέφει τιμή true ή false αν η παράμετρος μπορεί να αντιστοιχεί σε ονοματεπώνυμο υπαλλήλου ή όχι, αντίστοιχα.

1.2. Υλοποίηση

Οι παραδοχές υλοποίησης της ValidName() είναι οι εξής:

- [1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά
- [2] Ξεχωρίζουμε το όνομα από το επώνυμο με βάση τον χαρακτήρα κενό ' ', όπου ισχύουν οι ίδιες παραδοχές υλοποίησης και για τα δύο
 - [2.1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 3
 - [2.2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 15
 - [2.3] Πρέπει να περιέχει μόνο γράμματα
 - [2.4] Το 1ο γράμμα πρέπει να είναι κεφαλαίο
 - [2.5] Τα γράμματα εκτός από το 1ο, πρέπει να είναι πεζά
 - [2.6] Τα γράμματα πρέπει να είναι όλα λατινικά

2. bool ValidPassword(string Password)

2.1. Λειτουργία

Η συνάρτηση αυτή δέχεται ως παράμετρο ένα κείμενο και επιστρέφει τιμή true ή false αν η παράμετρος αποτελεί έναν αποδεκτό κωδικό πρόσβασης. Προϋποθέσεις κωδικού: α. Τουλάχιστον 12 χαρακτήρες, β. Συνδυασμός κεφαλαίων γραμμάτων, πεζών γραμμάτων, αριθμών και συμβόλων (τουλάχιστον έναν χαρακτήρα από το καθένα), γ. Τα γράμματα να είναι λατινικοί χαρακτήρες, δ. Να ξεκινάει από κεφαλαίο γράμμα και να τελειώνει με αριθμό.

2.2. Υλοποίηση

Οι παραδοχές υλοποίησης της ValidPassword() είναι οι εξής:

- [1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 12
- [2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 24
- [3] Συνδυασμός χαρακτήρων
 - [3.1] Να περιέχει τουλάχιστον 1 κεφαλαίο γράμμα
 - [3.2] Να περιέχει τουλάχιστον 1 πεζό γράμμα
 - [3.3] Να περιέχει τουλάχιστον 1 ψηφίο
 - [3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο
- [4] Να μην περιέχει χαρακτήρες διαφυγής
- [5] Τα γράμματα πρέπει να είναι λατινικοί χαρακτήρες
- [6] Πρέπει να ξεκινάει από κεφαλαίο γράμμα και να τελειώνει με ψηφίο
 - [6.1] Πρέπει να ξεκινάει από κεφαλαίο γράμμα
 - [6.2] Πρέπει να τελειώνει με ψηφίο

3. EncryptPassword(string Password, ref string EncryptedPW)

3.1. Λειτουργία

Η μέθοδος αυτή δέχεται ως παράμετρο έναν κωδικό πρόσβασης και επιστρέφει τον κρυπτογραφημένο κωδικό με βάση τον Κώδικα του Καίσαρα (Caesar's Cipher), με αλφάβητο το ASCII (128 χαρακτήρες, βλέπε [εδώ](#)) και ολίσθηση κατά 5 θέσεις. Το αποτέλεσμα αυτό επιστρέφεται μέσω της αντίστοιχης by ref παραμέτρου που δέχεται.

3.2. Υλοποίηση

Οι παραδοχές υλοποίησης της EncryptPassword() είναι οι εξής:

- [1] Ο κωδικός πρέπει να είναι έγκυρος σύμφωνα με τις [παραδοχές υλοποίησης](#) της ValidPassword()

4. CheckPhone(string Phone, ref int TypePhone, ref string InfoPhone)

4.1. Λειτουργία

Η μέθοδος αυτή δέχεται ως κείμενο έναν αριθμό και ελέγχει αν αντιστοιχεί σε τηλέφωνο. Τα αποτελέσματα επιστρέφονται μέσω των αντίστοιχων by ref παραμέτρων που δέχεται. Συγκεκριμένα, αν είναι σταθερό τηλέφωνο, επιστρέφει ως τύπο τηλεφώνου (TypePhone) μηδέν (0) και ως πληροφορίες τηλεφώνου (InfoPhone) τη ζώνη που ανήκει ο αριθμός τηλεφώνου (8 ζώνες, βλέπε [εδώ](#)). Αν είναι κινητό τηλέφωνο, επιστρέφει ως τύπο τον αριθμό ένα (1) και ως πληροφορίες την εταιρεία κινητής τηλεφωνίας (βλέπε [εδώ](#)). Διαφορετικά, αν δεν είναι έγκυρος αριθμός τηλεφώνου, επιστρέφει ως τύπο τον αριθμό -1 και null στις πληροφορίες.

4.2. Υλοποίηση

Οι παραδοχές υλοποίησης της CheckPhone() είναι οι εξής:

- [1] Να περιέχει μόνο αριθμούς
- [2] Οι αριθμοί να είναι ακριβώς 10
- [3] Να ξεκινάει σε 2 αν πρόκειται για σταθερό
 - [3.1] Να ανήκει σε ζώνη
- [4] Να ξεκινάει σε 69 αν πρόκειται για κινητό
 - [4.1] Να ανήκει σε εταιρία κινητής τηλεφωνίας

5. InfoEmployee(Employee EmpIX, ref int Age, ref int YearsOfExperience)

5.1. Λειτουργία

Η μέθοδος αυτή δέχεται ως παραμέτρους τα στοιχεία ενός υπαλλήλου, και επιστρέφει την ηλικία του και τα χρόνια προϋπηρεσίας στην εταιρεία. Τα αποτελέσματα αυτά επιστρέφονται μέσω των αντίστοιχων by ref παραμέτρων που δέχεται.

5.2. Υλοποίηση

Οι παραδοχές υλοποίησης της InfoEmployee() είναι οι εξής:

- [1] Η ηλικία πρέπει να είναι από 18-70 χρονών
- [2] Η ημερομηνία πρόσληψης πρέπει να είναι από την ημερομηνία γέννησης μεταγενέστερα κατά 18 χρόνια έως την τρέχουσα ημερομηνία

6. int LiveInAthens(Employee Empls)

6.1. Λειτουργία

Η συνάρτηση αυτή επιστρέφει το πλήθος των υπαλλήλων που κατοικούν στην Αθήνα. Η συγκεκριμένη συνάρτηση κάνει κλήση της συνάρτησης CheckPhone().

6.2. Υλοποίηση

Οι παραδοχές υλοποίησης της LiveInAthens() είναι οι εξής:

- [1] Το σταθερό τηλέφωνο πρέπει να ξεκινάει με "21" για τους κατοίκους της Αθήνας
- [1.1] Οι πληροφορίες τηλεφώνου πρέπει να είναι "Μητροπολιτική Περιοχή Αθήνας - Πειραιά" για τους κατοίκους της Αθήνας

B. Δημιουργία Περιπτώσεων Ελέγχου (TestCases)

1. ValidName Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	string[] subNames = Name.Split(' '); subNames.Length != 2	subNames.Length == 2	subNames.Length != 2
2	firstName = subNames[0]; lastName = subNames[1];		
2.1	firstName.Length < 3 OR lastName.Length < 3	firstName.Length >= 3 AND lastName.Length >= 3	firstName.Length < 3 OR lastName.Length < 3
2.2	firstName.Length > 15 OR lastName.Length > 15	firstName.Length <= 15 AND lastName.Length <= 15	firstName.Length > 15 OR lastName.Length > 15
2.3	NOT firstName.All(char.IsLetter) OR NOT lastName.All(char.IsLetter)	firstName.All(char.IsLetter) == true AND lastName.All(char.IsLetter) == true	firstName.All(char.IsLetter) == false OR lastName.All(char.IsLetter) == false
2.4	NOT char.IsUpper(firstName[0]) OR NOT char.IsUpper(lastName[0])	char.IsUpper(firstName[0]) == true AND char.IsUpper(lastName[0]) == true	char.IsUpper(firstName[0]) == false OR char.IsUpper(lastName[0]) == false
2.5	NOT firstName.Substring(1).All(char.IsLower) OR	firstName.Substring(1).All(char.IsLower) == true AND lastName.Substring(1).All(ch	firstName.Substring(1).All(char.IsLower) == false OR lastName.Substring(1).All(c

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

	NOT lastName.Substring(1).All(char.IsLower)	ar.IsLower) == true	har.IsLower) == false
2.6	char[] firstNameArray = firstName.ToCharArray(); char[] lastNameArray = lastName.ToCharArray(); Each Loop : NOT (firstNameArray[i] < 'A' OR firstNameArray[i] > 'Z') OR (firstNameArray[i] < 'a' OR firstNameArray[i] > 'z') OR Each Loop : NOT (lastNameArray[i] < 'A' OR lastNameArray[i] > 'Z') OR (lastNameArray[i] < 'a' OR lastNameArray[i] > 'z')	Each Loop : (firstNameArray[i] >= 'A' AND firstNameArray[i] <= 'Z') OR (firstNameArray[i] >= 'a' AND firstNameArray[i] <= 'z') AND Each Loop : (lastNameArray[i] >= 'A' AND lastNameArray[i] <= 'Z') OR (lastNameArray[i] >= 'a' AND lastNameArray[i] <= 'z')	Each Loop : (firstNameArray[i] < 'A' OR firstNameArray[i] > 'Z') OR (firstNameArray[i] < 'a' OR firstNameArray[i] > 'z') OR Each Loop : (lastNameArray[i] < 'A' OR lastNameArray[i] > 'Z') OR (lastNameArray[i] < 'a' OR lastNameArray[i] > 'z')

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	Name	Flag	Hint
1	Vasilis	false	firstNameAndSurname
2	Theoxaris	false	firstNameAndSurname
3	Om Alhaz	false	moreThan3Chars
4	Vasilis At	false	moreThan3Chars
5	Georgeeeeeeeeeee Theoxaris	false	lessThan15Chars
6	Omar Alhaaaaaaaaaaaaaz	false	lessThan15Chars
7	Vasilis123 Athanasiou	false	onlyLetters
8	George Theo_xaris	false	onlyLetters
9	omar Alhaz	false	firstLetterUpper
10	Vasilis athanasiou	false	firstLetterUpper
11	GeorGe Theoxaris	false	restLettersLower
12	Omar AlhAz	false	restLettersLower
13	Βασίλης Athanasiou	false	onlyLatin
14	George Θεοχάρης	false	onlyLatin
15	Omar Alhaz	true	validName
16	Vasilis Athanasiou	true	validName
17	George Theoxaris	true	validName
NameError_1	Vasileios Evangelos Athanasiou	true	firstNameAndSurname

Hint	Message
firstNameAndSurname	[1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά
moreThan3Chars	[2.1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 3
lessThan15Chars	[2.2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 15
onlyLetters	[2.3] Πρέπει να περιέχει μόνο γράμματα
firstLetterUpper	[2.4] Το 1ο γράμμα πρέπει να είναι κεφαλαίο
restLettersLower	[2.5] Τα γράμματα εκτός από το 1ο, πρέπει να είναι πεζά
onlyLatin	[2.6] Τα γράμματα πρέπει να είναι όλα λατινικά
validName	Έγκυρο ονοματεπώνυμο

2. ValidPassword Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	Password.Length < 12	Password.Length >= 12	Password.Length < 12
2	Password.Length > 24	Password.Length <= 24	Password.Length > 24
3	Συνδυασμός χαρακτήρων		
3.1	NOT Password.Any(char.IsUpper)	Password.Any(char.IsUpper) == true	Password.Any(char.IsUpper) == false
3.2	NOT Password.Any(char.IsLower)	Password.Any(char.IsLower) == true	Password.Any(char.IsLower) == false
3.3	NOT Password.Any(char.IsDigit)	Password.Any(char.IsDigit) == true	Password.Any(char.IsDigit) == false
3.4	NOT Password.Any(symbol => "!@#%^&*()_+ - =[]{} ;:\",.<>/?".Contains(symbol))	Password.Any(symbol => "!@#%^&*()_+ - =[]{} ;:\",.<>/?".Contains(symbol)) == true	Password.Any(symbol => "!@#%^&*()_+ - =[]{} ;:\",.<>/?".Contains(symbol)) == false
4	Password.Any(char.IsWhiteSpace)	Password.Any(char.IsWhiteSpace) == false	Password.Any(char.IsWhiteSpace) == true
5	char[] passwordArray = Password.ToCharArray(); Each Loop : NOT (passwordArray[i] < 'A' OR passwordArray[i] > 'Z') OR (passwordArray[i] < 'a' OR passwordArray[i] > 'z')	Each Loop : (passwordArray[i] >= 'A' AND passwordArray[i] <= 'Z') OR (passwordArray[i] < 'a' AND passwordArray[i] > 'z')	Each Loop : (passwordArray[i] < 'A' OR passwordArray[i] > 'Z') OR (passwordArray[i] < 'a' OR passwordArray[i] > 'z')
6	Πρέπει να ξεκινάει από κεφαλαίο γράμμα και να τελειώνει με ψηφίο		
6.1	NOT char.IsUpper>Password[0])	char.IsUpper>Password[0]) == true	char.IsUpper>Password[0]) == false
6.2	NOT char.IsDigit>Password[Password.Length - 1])	char.IsDigit>Password[Password.Length - 1]) == true	char.IsDigit>Password[Password.Length - 1]) == false

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	Password	Flag	Hint
1	Ako2	false	moreThan12Chars
2	Akkakbfpaoqweh!@#1224hhff1	false	lessThan24Chars
3	!@#%^&*^&^\$#@!	false	oneUpper + oneLower + oneDigit +

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

			one Symbol
4	george@12#po12	false	oneUpper
5	OMARGYPAS2!@13	false	oneLower
6	Ath@n@siouvasil	false	oneDigit
7	George2001theo13	false	oneSymbol
8	George!@#13 hf	false	noWhitespaces
9	Γιωργοςπαδα@2024	false	onlyLatin
10	omara#Alhaz2001	false	startsWithUpper
11	Koaos1^klpoplkjg	false	endsWithDigit
12	Georgethe8o@#\$9	true	validPassword
13	Vasilisatha8no#6	true	validPassword
14	Omar_@Alhaz123	true	validPassword
PasswordError_2	Pao131908~123	true	oneSymbol

Hint	Message
moreThan12Chars	[1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 12
lessThan24Chars	[2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 24
oneUpper	[3.1] Να περιέχει τουλάχιστον 1 κεφαλαίο γράμμα
oneLower	[3.2] Να περιέχει τουλάχιστον 1 πεζό γράμμα
oneDigit	[3.3] Να περιέχει τουλάχιστον 1 ψηφίο
oneSymbol	[3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο
noWhitespaces	[4] Να μην περιέχει χαρακτήρες διαφυγής
onlyLatin	[5] Τα γράμματα πρέπει να είναι λατινικοί χαρακτήρες
startsWithUpper	[6.1] Πρέπει να ξεκινάει από κεφαλαίο γράμμα
endsWithDigit	[6.2] Πρέπει να τελειώνει με ψηφίο
validPassword	Έγκυρος κωδικός πρόσβασης

3. EncryptPassword Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	ValidPassword>Password	ValidPassword(Passwor d) == true	ValidPassword(Passwor d) == false

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	Password	EncryptedPW	Hint
1	omar24323	null	invalidPassword
2	villys12345	null	invalidPassword
3	theodosis123	null	invalidPassword
4	Om	null	invalidPassword
5	Tigrhs_sdfdssdfsdfsdf@16723	null	invalidPassword
6	Τρούσσας_123	null	invalidPassword
7	G.Theo_@13223	L3YmjtdE68778	validPassword + validEncryption
8	Makh\$_@18923	Rfpm)dE6=>78	validPassword + validEncryption
9	Tango_@12093	YfsltdE675>8	validPassword + validEncryption
10	Lanaras_@16723	QfsfwfxdE6;<78	validPassword + validEncryption
11	George_!12525	Ljtwljd&67:7:	validPassword

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

			+ validEncryption
12	Krouska_123456	Pwtzxpfd6789;;	validPassword + validEncryption

Hint	Message
invalidPassword	[1] Ο κωδικός πρέπει να είναι έγκυρος σύμφωνα με τις παραδοχές υλοποίησης της ValidPassword()
validPassword	Έγκυρος κωδικός πρόσβασης
validEncryption	Σωστή κρυπτογράφηση

4. CheckPhone Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	NOT Phone.All(char.IsDigit)	Phone.All(char.IsDigit) == true	Phone.All(char.IsDigit) == false
2	Phone.Length != 10	Phone.Length == 10	Phone.Length != 10
3	Phone.StartsWith("2")	Phone.StartsWith("2") == true	Phone.StartsWith("2") == false
3.1	Phone[1] >= '1' AND Phone[1] <= '8'	Phone[1] >= '1' AND Phone[1] <= '8'	Phone[1] < 1 OR Phone[1] > 8
4	Phone.StartsWith("69")	Phone.StartsWith("69") == true	Phone.StartsWith("69") == false
4.1	Phone[2] == 0 OR (Phone[2] >= 3 AND Phone[2] <= 5) OR (Phone[2] >= 7 AND Phone[2] <= 9)	Phone[2] == 0 OR (Phone[2] >= 3 AND Phone[2] <= 5) OR (Phone[2] >= 7 AND Phone[2] <= 9)	Phone[2] != 0 AND (Phone[2] < 3 OR Phone[2] > 5) AND (Phone[2] < 7 OR Phone[2] > 9)

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα		
#id	Phone	Type Phone	Info Phone	Hint
1	210-124567	-1	null	onlyDigits
2	690A49898c	-1	null	onlyDigits
3	22314	-1	null	digits10
4	69312	-1	null	digits10
5	23444444444444	-1	null	digits10
6	69988888888888	-1	null	digits10
7	140124567	-1	null	startsWith2HomePhone
8	2001010101	-1	null	belongsToZone
9	2101010101	0	Metropolitan Area of Athens - Piraeus	validHomePhone + zone1
10	2201010101	0	Eastern Central Greece, Attica, Aegean Islands	validHomePhone + zone2
11	2301010101	0	Central Macedonia	validHomePhone + zone3

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

12	2401010101	0	Thessaly, Western Macedonia	validHomePhone + zone4
13	2501010101	0	Thrace, Eastern Macedonia	validHomePhone + zone5
14	2601010101	0	Epirus, Western Central Greece, Western Peloponnese, Ionian Islands	validHomePhone + zone6
15	2701010101	0	Eastern Peloponnese, Kythera	validHomePhone + zone7
16	2801010101	0	Crete	validHomePhone + zone8
17	2901010101	-1	null	belongsToZone
18	5971476763	-1	null	startsWith69MobilePhone
19	6900101010	1	Nova	validMobilePhone + nova
20	6910101010	-1	null	belongsToMobileDataComp
21	6920101010	-1	null	belongsToMobileDataComp
22	6930101010	1	Nova	validMobilePhone + nova
23	6940101010	1	Vodafone	validMobilePhone + vodafone
24	6950101010	1	Vodafone	validMobilePhone + vodafone
25	6960101010	-1	null	belongsToMobileDataComp
26	6970101010	1	Cosmote	validMobilePhone + cosmote
27	6980101010	1	Cosmote	validMobilePhone + cosmote
28	6990101010	1	Nova	validMobilePhone + nova
PhoneError_3	210 28 12 967	0	Metropolitan Area of Athens - Piraeus	onlyDigits

Hint	Message
onlyDigits	[1] Να περιέχει μόνο αριθμούς
digits10	[2] Οι αριθμοί να είναι ακριβώς 10
startsWith2HomePhone	[3] Να ξεκινάει σε 2 αν πρόκειται για σταθερό
belongsToZone	[3.1] Να ανήκει σε ζώνη
validHomePhone	Έγκυρο σταθερό τηλέφωνο
zone1	με ζώνη την Μητροπολιτική Περιοχή Αθήνας - Πειραιά
zone2	με ζώνη την Ανατολική Στερεά Ελλάδα, Αττική, Νησιά Αιγαίου
zone3	με ζώνη την Κεντρική Μακεδονία
zone4	με ζώνη την Θεσσαλία, Δυτική Μακεδονία
zone5	με ζώνη την Θράκη, Ανατολική Μακεδονία
zone6	με ζώνη την Ήπειρο, Δυτική Στερεά Ελλάδα, Δυτική Πελοπόννησο, Ιόνια Νησιά
zone7	με ζώνη την Ανατολική Πελοπόννησο, Κύθηρα
zone8	με ζώνη την Κρήτη
startsWith69MobilePhone	[4] Να ξεκινάει σε 69 αν πρόκειται για κινητό
belongsToMobileDataComp	[4.1] Να ανήκει σε εταιρία κινητής τηλεφωνίας
validMobilePhone	Έγκυρο κινητό τηλέφωνο
nova	με εταιρία κινητής τηλεφωνίας τη Nova
cosmote	με εταιρία κινητής τηλεφωνίας τη Cosmote
vodafone	με εταιρία κινητής τηλεφωνίας τη Vodafone

5. InfoEmployee Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	EmplX.Birthday >= Today - 70 AND EmplX.Birthday <= Today - 18	EmplX.Birthday >= Today - 70 AND EmplX.Birthday <= Today - 18	EmplX.Birthday < Today - 70 OR EmplX.Birthday > Today - 18
2	EmplX.HiringDate >= EmplX.Birthday.AddYears(18) AND EmplX.HiringDate <= DateTime.Today	EmplX.HiringDate >= EmplX.Birthday.AddYears(18) AND EmplX.HiringDate <= DateTime.Today	EmplX.HiringDate < EmplX.Birthday.AddYears(18) OR EmplX.HiringDate > DateTime.Today

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
empl1	George Theocharis	2102322751	6998843565	2001-08-23	2022-05-08
empl2	Panagiotis Petropoulos	2102887987	6975522693	1985-04-03	2010-06-15
empl3	Vasilis Athanasiou	2201010101	6980101010	1999-07-15	2020-08-15
empl4	Evangelos Tsimentas	2701010101	6990101010	1990-03-04	2015-06-03
empl5	Omar Alhaz	2301010101	6970101010	2000-02-01	2020-05-06
empl6	Akrivh Krouska	2301110101	6970101011	2000-02-01	2020-05-06
empl7	Pantelhs Tatsis	2601010101	6975522897	2001-01-08	2020-01-05
empl8	Aristotelis Sifakis	2601010801	6973298777	2001-01-07	2019-05-08
empl9	Antonis Kokkinos	2102322751	6978899987	2001-02-13	2021-05-02
empl10	Katerina Vossoy	2105042087	6987304875	2001-07-11	2022-09-01
empl11	Athina Nikolaou	2700101010	6975588489	1997-02-27	2023-05-22
empl12	Aggelikh Panou	2401010101	6954848497	1998-04-30	2019-06-07
empl13	Hlias Konstantinidis	2701010102	6935660423	1950-09-25	1970-04-08
empl14	George Gomez	2501010101	6990101010	2007-12-01	2023-08-01
empl15	George Santos	2105000000	6988888888	1990-04-29	2014-06-01
empl16	Antonis Varvaris	2300000000	6955555555	2000-01-10	2018-05-06
empl17	Xristos Pilarinos	2105555555	6987451235	1999-05-21	2019-06-30
emplfault1	Aggelos Skandalis	2501010102	6988558589	1953-05-30	1977-02-03
emplfault2	Dimitris Siametis	2104588989	6999999999	2006-01-02	2024-01-01

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα		
#id	Employee	Age	Years Of Experience	Hint
1	empl1	24	1	validAge + "22" + validXpYears + "1"
2	empl2	38	13	validAge + "38" + validXpYears + "13"
3	empl3	24	3	validAge + "24" + validXpYears + "3"
4	empl4	33	8	validAge + "33" + validXpYears + "8"

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

5	empl5	24	3	validAge + "24" + validXpYears + "3"
6	empl6	24	3	validAge + "24" + validXpYears + "3"
7	empl7	23	4	validAge + "23" + validXpYears + "4"
8	empl8	23	4	validAge + "23" + validXpYears + "4"
9	empl9	23	2	validAge + "23" + validXpYears + "2"
10	empl10	22	1	validAge + "22" + validXpYears + "1"
11	empl11	26	0	validAge + "26" + validXpYears + "0"
12	empl12	25	4	validAge + "25" + validXpYears + "4"
13	empl13	-1	53	ageBetween18And70
14	empl14	-1	-1	youngerThan18ForHiring
15	empl15	33	9	validAge + "33" + validXpYears + "9"
16	empl16	24	5	validAge + "24" + validXpYears + "5"
17	empl17	24	4	validAge + "24" + validXpYears + "4"
AgeError_4	emplFault1	70	47	ageBetween18And70
YearsOfXpError_5	emplFault2	18	0	youngerThan18ForHiring

Hint	Message
ageBetween18And70	[1] Η ηλικία πρέπει να είναι από 18-70 χρονών
youngerThan18ForHiring	[2] Η ημερομηνία πρόσληψης πρέπει να είναι από την ημερομηνία γέννησης μεταγενέστερα κατά 18 χρόνια έως την τρέχουσα ημερομηνία
validAge	Έγκυρη ηλικία :
validXpYears	Έγκυρα χρόνια προϋπηρεσίας :

6. LiveInAthens Test Case

id	Συνθήκη Εισόδου	Κλάσεις ισοδύναμων τιμών	
		Έγκυρες	Άκυρες
1	CheckPhone(emp.HomePhone, ref Typephone, ref Infophone)	TypePhone == 0 AND InfoPhone != null	TypePhone != 0 AND InfoPhone == null
1.1	InfoPhone == "Metropolitan Area of Athens - Piraeus"	InfoPhone == "Metropolitan Area of Athens - Piraeus"	InfoPhone != "Metropolitan Area of Athens - Piraeus"

Λίστα Υπαλλήλων empl1				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2202625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2302324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2401234681	6945678789	2001-03-24	2023-12-20

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Λίστα Υπαλλήλων emp1s2

Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Tom Hanks	2102625345	6971345456	1959-07-09	1989-06-09
Leonardo DiCaprio	2102324901	6981314145	1974-11-11	1995-03-15
Meryl Streep	2801234681	6945678789	1955-06-22	1975-05-05
Robert Downey	2102625345	6971345456	1965-04-04	1984-08-23
Johnny Depp	2102324901	6981314145	1963-06-09	1984-09-10
Angelina Jolie	2801234681	6945678789	1975-06-04	1994-10-12
Chris Hemsworth	2102324901	6981314145	1983-08-11	2002-03-18

Λίστα Υπαλλήλων emp1s3

Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Brad Pitt	2702625345	6971345456	1963-12-18	1987-05-20
Jennifer Aniston	2602324901	6981314145	1969-02-11	1992-07-15
Tom Cruise	2801234681	6945678789	1962-07-03	1981-10-10
George Clooney	2102625345	6971345456	1961-05-06	1982-09-22
Julia Roberts	2102324901	6981314145	1967-10-28	1988-07-15
Denzel Washington	2801234681	6945678789	1954-12-28	1978-06-12
Scarlett Johansson	2102625345	6971345456	1984-11-22	2003-05-10
Robert Pattinson	2801234681	6945678789	1986-05-13	2004-11-07

Λίστα Υπαλλήλων emp1s4

Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2102625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2102324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2801234681	6945678789	2001-03-24	2023-12-20
Christos Troussas	2131414567	6941296963	1994-11-15	2024-01-01
Akrivi Krouska	2215678901	6901234345	1998-05-19	2023-12-20
Ioannis Vogiatzis	2801234681	6945678789	2001-03-24	2023-12-20
George Meletiou	2461345679	6931345450	1980-06-04	2023-11-03
Panagiotis Giannakopoulos	2151234510	6901234342	1978-06-10	2024-02-20
Stavros Fatouros	2612828456	6951616987	1990-01-24	2023-03-11

Λίστα Υπαλλήλων emp1s5

Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Bartłomiej Dragowski	2145678901	6905678901	1997-08-19	2024-01-19
Tin Jedvaj	2156789012	6935678901	1995-11-28	2023-07-14
Filip Mladenovic	2167890123	6995678901	1991-08-15	2023-07-01
Giannis Kotsiras	2178901234	6945678901	1992-12-16	2021-07-01
Willian Arao	2189012345	6955678901	1992-03-12	2023-08-17
Carlos Zeca	2190123456	6975678901	1988-08-21	2023-07-01
Bernard Duarte	2101234567	6985678901	1992-09-08	2022-08-21
Anastasios Bakasetas	2112345678	6935678901	1993-06-28	2024-01-22
Adam Cerin	2123456789	6945678901	1999-07-16	2022-07-02
Sebastian Palacios	2134567890	6975678901	1992-01-20	2021-08-31
Fotis Ioannidis	2145678901	6985678901	2000-01-10	2020-08-10

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Λίστα Υπαλλήλων empl6				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2102625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2402324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2501234681	6945678789	2001-03-24	2023-12-20
Dimitra Papadopoulou	2604567890	6934567890	2000-01-15	2023-06-20
Thanasis Papanikolaou	2705678901	6945678901	1999-11-28	2019-03-05
Eirini Nikolaou	2806789012	6956789012	2001-09-08	2023-08-12
Panagiotis Georgopoulos	2107890123	6907890123	2000-07-22	2019-01-01

Λίστα Υπαλλήλων empl7				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Despina Vandí	2201234567	6931234567	1964-07-22	1998-10-01
Sakis Rouvas	2102345678	6942345678	1972-01-05	1990-05-15
Kostas Martakis	2703456789	6953456789	1974-09-14	2000-12-30
Elena Paparizou	2804567890	6944567890	1982-01-31	2010-02-20
Giorgos Liagas	2605678901	6975678901	1974-10-26	2001-07-10

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	List Of Employees	Flag	Hint
1	empls1	0	numAthensEmps + "0/3"
2	empls2	5	numAthensEmps + "5/7"
3	empls3	3	numAthensEmps + "3/8"
4	empls4	4	numAthensEmps + "3/9"
5	empls5	11	numAthensEmps + "11/11"
6	empls6	2	numAthensEmps + "3/7"
7	empls7	1	numAthensEmps + "1/5"

Hint	Message
numAthensEmps	Υπάλληλοι που κατοικούν στην Αθήνα :

Γ. Ανάπτυξη Ελέγχων Μονάδων (UnitTests)

1. ValidName Unit Test

```
[TestMethod]
public void TestMethodValidName()
{
    // Παραδοχές υλοποίησης
    string firstnameAndSurname = "[1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά";
```


ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
string moreThan3Chars = " [2.1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 3";
string lessThan15Chars = " [2.2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 15";
string onlyLetters = " [2.3] Πρέπει να περιέχει μόνο γράμματα";
string firstLetterUpper = " [2.4] Το 1ο γράμμα πρέπει να είναι κεφαλαίο";
string restLettersLower = " [2.5] Τα γράμματα εκτός από το 1ο, πρέπει να είναι πεζά";
string onlyLatin = " [2.6] Τα γράμματα πρέπει να είναι όλα λατινικά";
string validName = " Έγκυρο ονοματεπώνυμο";
```

```
// Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάρουμε
HRLib.Personnel per = new HRLib.Personnel();
```

```
// Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)
```

```
object[,] testcases =
{
// { id,          "Ονοματεπώνυμο",          εκτιμώμενη τιμή          "Παραδοχή υλοποίησης"}
//                                     επιστροφής της ValidName()
  { "1",          "Vasilis",          false,          firstnameAndSurname },
  { "2",          "Theoxaris",          false,          firstnameAndSurname },
  { "3",          "Om Alhaz",          false,          moreThan3Chars },
  { "4",          "Vasilis At",          false,          moreThan3Chars },
  { "5",          "Georgeeeeeeeeeee Theoxaris",          false,          lessThan15Chars },
  { "6",          "Omar Alhaaaaaaaaaaaaaaz",          false,          lessThan15Chars },
  { "7",          "Vasilis123 Athanasiou",          false,          onlyLetters },
  { "8",          "George Theo_xaris",          false,          onlyLetters },
  { "9",          "omar Alhaz",          false,          firstLetterUpper },
  { "10",         "Vasilis athanasiou",          false,          firstLetterUpper },
  { "11",         "GeorGe Theoxaris",          false,          restLettersLower },
  { "12",         "Omar AlhAz",          false,          restLettersLower },
  { "13",         "Βασίλης Athanasiou",          false,          onlyLatin },
  { "14",         "George Θεοχάρης",          false,          onlyLatin },
  { "15",         "Omar Alhaz",          true,          validName },
  { "16",         "Vasilis Athanasiou",          true,          validName },
  { "17",         "George Theoxaris",          true,          validName },
  { "NameError_1", "Vasileios Evangelos Athanasiou",          true,          firstnameAndSurname }
};
```

```
// Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)
```

```
int i = 0;
bool failed = false;
```

```
// Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου
```

```
for (i = 0; i < testcases.GetLength(0); i++)
```

```
// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases
```

```
{
    try
    {
        // Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,
        // δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases
        string Name = (string)testcases[i, 1];
        bool expectedValue = (bool)testcases[i, 2]; // Η τιμή που περιμένω να επιστρέψει η μέθοδος ValidName()
        για την ημερομηνία i
        bool actualValue = per.ValidName(Name); // Η τιμή που επιστρέφει η μέθοδος ValidName() για την
        ημερομηνία i
        Assert.AreEqual(expectedValue, actualValue);
    }
    catch (Exception e)
    {
        // Απέτυχε το Test Case
        failed = true;
        // Καταγράφουμε το Test Case που απέτυχε
        Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξαίρεση: {2} ",
            (string)testcases[i, 0], (string)testcases[i, 3], e.Message);
    }
}
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου ελέγχου,
Μήνυμα του exception
}
}

// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception
if (failed)
    Assert.Fail();
}
```

2. ValidPassword Unit Test

```
[TestMethod]
public void TestMethodValidPassword()
{
    // Παραδοχές υλοποίησης
    string moreThan12Chars = "[1] Το πλήθος των χαρακτήρων πρέπει να είναι τουλάχιστον 12";
    string lessThan24Chars = "[2] Το πλήθος των χαρακτήρων πρέπει να είναι το πολύ 24";
    string oneUpper = "[3.1] Να περιέχει τουλάχιστον 1 κεφαλαίο γράμμα";
    string oneLower = "[3.2] Να περιέχει τουλάχιστον 1 πεζό γράμμα";
    string oneDigit = "[3.3] Να περιέχει τουλάχιστον 1 ψηφίο";
    string oneSymbol = "[3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο";
    string noWhitespaces = "[4] Να μην περιέχει χαρακτήρες διαφυγής";
    string onlyLatin = "[5] Τα γράμματα πρέπει να είναι λατινικοί χαρακτήρες";
    string startsWithUpper = "[6.1] Πρέπει να ξεκινάει από κεφαλαίο γράμμα";
    string endsWithDigit = "[6.2] Πρέπει να τελειώνει με ψηφίο";
    string validPassword = "Έγκυρος κωδικός πρόσβασης";

    // Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάρουμε
    HRLib.Personnel per = new HRLib.Personnel();

    // Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)
    object[,] testcases =
    {
        // { id, "Κωδικός", εκτιμώμενη τιμή "Παραδοχή υλοποίησης"
        // επιστροφής της ValidPassword()
        { "1", "Ako2", false, moreThan12Chars },
        { "2", "Akkakbfpaoweh!@#1224hhff1", false, lessThan24Chars },
        { "3", "!@#$%^&**&^%$#@!", false, oneUpper + oneLower + oneDigit
+ oneSymbol },
        { "4", "george@12#po12", false, oneUpper },
        { "5", "OMARGYPAS2!@13", false, oneLower },
        { "6", "Ath@n@siouvasil", false, oneDigit },
        { "7", "George2001theo13", false, oneSymbol },
        { "8", "George!@#13 hf", false, noWhitespaces },
        { "9", "Γιωργοςπαδα@2024", false, onlyLatin },
        { "10", "omara#Alhaz2001", false, startsWithUpper },
        { "11", "Koaos1^klpoplkjg", false, endsWithDigit },
        { "12", "Georgethe8o@#$9", true, validPassword },
        { "13", "Vasilisatha8no#6", true, validPassword },
        { "14", "Omar _Alhaz123", true, validPassword },
        { "PasswordError_2", "Pao131908~123", true, oneSymbol }
    };

    // Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)
    int i = 0;
    bool failed = false;

    // Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
for (i = 0; i < testcases.GetLength(0); i++)
// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases
{
    try
    {
        // Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,
        // δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases
        string Password = (string)testcases[i, 1];
        bool expectedValue = (bool)testcases[i, 2]; // Η τιμή που περιμένω να επιστρέψει η μέθοδος
ValidPassword() για την ημερομηνία i
        bool actualValue = per.ValidPassword(Password); // Η τιμή που επιστρέφει η μέθοδος
ValidPassword() για την ημερομηνία i
        Assert.AreEqual(expectedValue, actualValue);
    }
    catch (Exception e)
    {
        // Απέτυχε το Test Case
        failed = true;
        // Καταγράφουμε το Test Case που απέτυχε
        Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξάιρεση: {2} ",
            (string)testcases[i, 0], (string)testcases[i, 3], e.Message);
        // id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου
        ελέγχου, Μήνυμα του exception
    }
}

// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception.
if (failed)
    Assert.Fail();
}
```

3. EncryptPassword Unit Test

```
[TestMethod]
public void TestMethodEncryptPassword()
{
    // Παραδοχές υλοποίησης
    string invalidPassword = " [1] Ο κωδικός πρέπει να είναι έγκυρος σύμφωνα με τις παραδοχές υλοποίησης της
ValidPassword()";
    string validPassword = " Έγκυρος κωδικός πρόσβασης";
    string validEncryption = " Σωστή κρυπτογράφηση";

    // Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάρουμε
    HRLib.Personnel per = new HRLib.Personnel();

    // Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)
    object[,] testcases =
    {
        // { id, "Κωδικός", εκτιμώμενη τιμή "Παραδοχή υλοποίησης" }
        // { id, "Κωδικός", επιστροφής της EncryptPassword()
        { "1", "omar24323", null, invalidPassword },
        { "2", "villys12345", null, invalidPassword },
        { "3", "theodosis123", null, invalidPassword },
        { "4", "Om", null, invalidPassword },
        { "5", "Tigrhs_sdfdssdfsdfsdfsdf@16723", null, invalidPassword },
        { "6", "Τρούσσας_123", null, invalidPassword },
        { "7", "G.Theo_@13223", "L3YmjtdE68778", validPassword +
validEncryption },
        { "8", "Makh$_@18923", "Rfpm)dE6=>78", validPassword +
validEncryption },
    }
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
        { "9",          "Tango_@12093",          "YfsItE675>8",          validPassword + validEncryption
    },
    { "10",          "Lanaras_@16723",          "QfsfwfxdE6;<78",          validPassword +
validEncryption },
    { "11",          "George_!12525",          "Ljtwljd&67:7:",          validPassword + validEncryption
    },
    { "12",          "Krouska_123456",          "Pwtzxpfd6789;:",          validPassword +
validEncryption }
};
```

// Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)

int i = 0;

bool failed = false;

// Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου

for (i = 0; i < testcases.GetLength(0); i++)

// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases

{

try

{

string TestcasePW = (string)testcases[i, 1]; // Ο κωδικός του testcase i

string ExpectedEnPW = (string)testcases[i, 2]; // Ο κρυπτογραφημένος κωδικός του testcase i που

περιμένω να επιστρέψει η EncryptPassword()

// Δήλωση και αρχικοποίηση ref μεταβλητών μεθόδου

string ActualEnPW = "";

per.EncryptPassword(TestcasePW, ref ActualEnPW);

// Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,

// δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases

Assert.AreEqual(ExpectedEnPW, ActualEnPW);

}

catch (Exception e)

{

// Απέτυχε το Test Case

failed = true;

// Καταγράφουμε το Test Case που απέτυχε

Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξαίρεση: {2} ",

(string)testcases[i, 0], (string)testcases[i, 3],

e.Message);

// id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου ελέγχου,

Μήνυμα του exception

}

// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception.

if (failed)

Assert.Fail();

}

4. CheckPhone Unit Test

[TestMethod]

public void TestMethodCheckPhone()

{

// Παραδοχές υλοποίησης

string onlyDigits = " [1] Να περιέχει μόνο αριθμούς";

string digits10 = " [2] Οι αριθμοί να είναι ακριβώς 10";

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
string startsWith2HomePhone = "[3] Να ξεκινάει σε 2 αν πρόκειται για σταθερό";
string belongsToZone = "[3.1] Να ανήκει σε ζώνη";
string validHomePhone = "Έγκυρο σταθερό τηλέφωνο";
string zone1 = " με ζώνη την Μητροπολιτική Περιοχή Αθήνας - Πειραιά";
string zone2 = " με ζώνη την Ανατολική Στερεά Ελλάδα, Αττική, Νησιά Αιγαίου";
string zone3 = " με ζώνη την Κεντρική Μακεδονία";
string zone4 = " με ζώνη την Θεσσαλία, Δυτική Μακεδονία";
string zone5 = " με ζώνη την Θράκη, Ανατολική Μακεδονία";
string zone6 = " με ζώνη την Ήπειρο, Δυτική Στερεά Ελλάδα, Δυτική Πελοπόννησο, Ιόνια Νησιά";
string zone7 = " με ζώνη την Ανατολική Πελοπόννησο, Κύθηρα";
string zone8 = " με ζώνη την Κρήτη";
string startsWith69MobilePhone = "[4] Να ξεκινάει σε 69 αν πρόκειται για κινητό";
string belongsToMobileDataComp = "[4.1] Να ανήκει σε εταιρία κινητής τηλεφωνίας";
string validMobilePhone = "Έγκυρο κινητό τηλέφωνο";
string nova = " με εταιρία κινητής τηλεφωνίας τη Nova";
string cosmote = " με εταιρία κινητής τηλεφωνίας τη Cosmote";
string vodafone = " με εταιρία κινητής τηλεφωνίας τη Vodafone";
```

```
// Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάρουμε
HRLib.Personnel per = new HRLib.Personnel();
```

```
// Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)
```

```
object[,] testcases =
{
    // { id,                "Αριθμός Τηλεφώνου",    εκτιμώμενη τιμή επιστροφής    εκτιμώμενη τιμή
    επιστροφής            "Παραδοχή υλοποίησης" }
    //                      της CheckPhone() στην TypePhone  της CheckPhone() στην InfoPhone
    { "1",                "210-124567",            -1,                            null,
onlyDigits },
    { "2",                "690A49898c",            -1,                            null,
onlyDigits },
    { "3",                "22314",                -1,                            null,                                digits10
},
    { "4",                "69312",                -1,                            null,                                digits10
},
    { "5",                "23444444444444",        -1,                            null,
digits10 },
    { "6",                "69988888888888",        -1,                            null,
digits10 },
    { "7",                "140124567",            -1,                            null,
startsWith2HomePhone },
    { "8",                "2001010101",            -1,                            null,
belongsToZone },
    { "9",                "2101010101",            0,                            "Metropolitan Area of Athens - Piraeus",
validHomePhone + zone1 },
    { "10",               "2201010101",            0,                            "Eastern Central Greece, Attica, Aegean
Islands",                validHomePhone + zone2 },
    { "11",               "2301010101",            0,                            "Central Macedonia",
validHomePhone + zone3 },
    { "12",               "2401010101",            0,                            "Thessaly, Western Macedonia",
validHomePhone + zone4 },
    { "13",               "2501010101",            0,                            "Thrace, Eastern Macedonia",
validHomePhone + zone5 },
    { "14",               "2601010101",            0,                            "Epirus, Western Central Greece, Western
Peloponnese, Ionian Islands", validHomePhone + zone6 },
    { "15",               "2701010101",            0,                            "Eastern Peloponnese, Kythera",
validHomePhone + zone7 },
    { "16",               "2801010101",            0,                            "Crete",
validHomePhone + zone8 },
    { "17",               "2901010101",            -1,                            null,
belongsToZone },
    { "18",               "5971476763",            -1,                            null,
startsWith69MobilePhone },
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

        { "19",          "6900101010",          1,          "Nova",
validMobilePhone + nova },
        { "20",          "6910101010",          -1,          null,
belongsToMobileDataComp },
        { "21",          "6920101010",          -1,          null,
belongsToMobileDataComp },
        { "22",          "6930101010",          1,          "Nova",
validMobilePhone + nova },
        { "23",          "6940101010",          1,          "Vodafone",
validMobilePhone + vodafone },
        { "24",          "6950101010",          1,          "Vodafone",
validMobilePhone + vodafone },
        { "25",          "6960101010",          -1,          null,
belongsToMobileDataComp },
        { "26",          "6970101010",          1,          "Cosmote",
validMobilePhone + cosmote },
        { "27",          "6980101010",          1,          "Cosmote",
validMobilePhone + cosmote },
        { "28",          "6990101010",          1,          "Nova",
validMobilePhone + nova },
        { "PhoneError_3", "210 28 12 967",          0,          "Metropolitan Area of Athens -
Piraeus",          onlyDigits }
};

```

// Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)

int i = 0;

bool failed = false;

// Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου

for (i = 0; i < testcases.GetLength(0); i++)

// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases

{

try

{

string TestcasePhone = (string)testcases[i, 1]; // Το τηλέφωνο του testcase i

int ExpectedTypePhone = (int)testcases[i, 2]; // Ο τύπος τηλεφώνου του testcase i που περιμένω να

επιστρέψει η CheckPhone()

string ExpectedInfoPhone = (string)testcases[i, 3]; // Οι πληροφορίες τηλεφώνου του testcase i που

περιμένω να επιστρέψει η CheckPhone()

// Δήλωση και αρχικοποίηση ref μεταβλητών μεθόδου

int ActualTypePhone = 100;

string ActualInfoPhone = "";

per.CheckPhone(TestcasePhone, ref ActualTypePhone, ref ActualInfoPhone);

// Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,

// δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases

Assert.AreEqual(ExpectedTypePhone, ActualTypePhone);

Assert.AreEqual(ExpectedInfoPhone, ActualInfoPhone);

}

catch (Exception e)

{

// Απέτυχε το Test Case

failed = true;

// Καταγράφουμε το Test Case που απέτυχε

Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξαίρεση: {2} ",
(string)testcases[i, 0], (string)testcases[i, 4], e.Message);

// id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου ελέγχου,

Μήνυμα του exception

}

}

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception.  
if (failed)  
    Assert.Fail();  
}
```

5. InfoEmployee Unit Test

```
[TestMethod]  
public void TestMethodInfoEmployee()  
{  
    // Παραδοχές υλοποίησης  
    string ageBetween18And70 = "[1] Η ηλικία πρέπει να είναι από 18-70 χρονών";  
    string youngerThan18ForHiring = "[2] Η ημερομηνία πρόσληψης πρέπει να είναι από την ημερομηνία  
γέννησης μεταγενέστερα κατά 18 χρόνια έως την τρέχουσα ημερομηνία";  
    string validAge = "Έγκυρη ηλικία : ";  
    string validXpYears = "Έγκυρα χρόνια προϋπηρεσίας : ";  
  
    // Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάrouμε  
    HRLib.Personnel per = new HRLib.Personnel();  
  
    // "Όνοματεπώνυμο", "Σταθερό Τηλέφωνο", "Κινητό Τηλέφωνο", "Ημερομηνία  
Γέννησης", "Ημερομηνία Πρόσληψης"  
    Employee empl1 = new Employee("George Theocharis", "2102322751", "6998843565",  
new DateTime(2001, 08, 23), new DateTime(2022, 05, 08));  
    Employee empl2 = new Employee("Panagiotis Petropoulos", "2102887987", "6975522693",  
new DateTime(1985, 04, 03), new DateTime(2010, 06, 15));  
    Employee empl3 = new Employee("Vasilis Athanasiou", "2201010101", "6980101010", new  
DateTime(1999, 07, 15), new DateTime(2020, 08, 15));  
    Employee empl4 = new Employee("Evangelos Tsimentas", "2701010101", "6990101010",  
new DateTime(1990, 03, 04), new DateTime(2015, 06, 03));  
    Employee empl5 = new Employee("Omar Alhaz", "2301010101", "6970101010", new  
DateTime(2000, 02, 01), new DateTime(2020, 05, 06));  
    Employee empl6 = new Employee("Akrivh Krouska", "2301110101", "6970101011", new  
DateTime(2000, 02, 01), new DateTime(2020, 05, 06));  
    Employee empl7 = new Employee("Pantelhs Tatsis", "2601010101", "6975522897", new  
DateTime(2001, 01, 08), new DateTime(2020, 01, 05));  
    Employee empl8 = new Employee("Aristotelis Sifakis", "2601010801", "6973298777", new  
DateTime(2001, 01, 07), new DateTime(2019, 05, 08));  
    Employee empl9 = new Employee("Antonis Kokkinos", "2102322751", "6978899987", new  
DateTime(2001, 02, 13), new DateTime(2021, 05, 02));  
    Employee empl10 = new Employee("Katerina Vossoy", "2105042087", "6987304875", new  
DateTime(2001, 07, 11), new DateTime(2022, 09, 01));  
    Employee empl11 = new Employee("Athina Nikolaou", "2700101010", "6975588489", new  
DateTime(1997, 02, 27), new DateTime(2023, 05, 22));  
    Employee empl12 = new Employee("Aggelikh Panou", "2401010101", "6954848497", new  
DateTime(1998, 04, 30), new DateTime(2019, 06, 07));  
    Employee empl13 = new Employee("Hlias Konstantinidis", "2701010102", "6935660423", new  
DateTime(1950, 09, 25), new DateTime(1970, 04, 08));  
    Employee empl14 = new Employee("George Gomez", "2501010101", "6990101010", new  
DateTime(2007, 12, 01), new DateTime(2023, 08, 01));  
    Employee empl15 = new Employee("George Santos", "2105000000", "6988888888", new  
DateTime(1990, 04, 29), new DateTime(2014, 06, 01));  
    Employee empl16 = new Employee("Antonis Varvaris", "2300000000", "6955555555", new  
DateTime(2000, 01, 10), new DateTime(2018, 05, 06));  
    Employee empl17 = new Employee("Xristos Pilarinos", "2105555555", "6987451235", new  
DateTime(1999, 05, 21), new DateTime(2019, 06, 30));  
    Employee emplfault1 = new Employee("Aggelos Skandalis", "2501010102", "6988558589",  
new DateTime(1953, 05, 30), new DateTime(1977, 02, 03));  
    Employee emplfault2 = new Employee("Dimitris Siametis", "2104588989", "6999999999", new  
DateTime(2006, 01, 02), new DateTime(2024, 01, 01));  
}
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)
object[,] testcases =
{
    // { id, "Υπάλληλος", εκτιμώμενη τιμή επιστροφής, εκτιμώμενη τιμή επιστροφής,
    "Παραδοχή υλοποίησης" }
    // της InfoEmployee() στην Age της InfoEmployee() στην YearsOfExperience
    { "1", empl1, 22, 1, validAge + "22"
+ validXpYears + "1" },
    { "2", empl2, 38, 13, validAge + "38"
+ validXpYears + "13" },
    { "3", empl3, 24, 3, validAge + "24"
+ validXpYears + "3" },
    { "4", empl4, 33, 8, validAge + "33"
+ validXpYears + "8" },
    { "5", empl5, 24, 3, validAge + "24"
+ validXpYears + "3" },
    { "6", empl6, 24, 3, validAge + "24"
+ validXpYears + "3" },
    { "7", empl7, 23, 4, validAge + "23"
+ validXpYears + "4" },
    { "8", empl8, 23, 4, validAge + "23"
+ validXpYears + "4" },
    { "9", empl9, 23, 2, validAge + "23"
+ validXpYears + "2" },
    { "10", empl10, 22, 1, validAge + "22"
+ validXpYears + "1" },
    { "11", empl11, 27, 0, validAge + "26"
+ validXpYears + "0" },
    { "12", empl12, 25, 4, validAge + "25"
+ validXpYears + "4" },
    { "13", empl13, -1, 53,
ageBetween18And70 },
    { "14", empl14, -1, -1,
youngerThan18ForHiring },
    { "15", empl15, 33, 9, validAge + "33"
+ validXpYears + "9" },
    { "16", empl16, 24, 5, validAge + "24"
+ validXpYears + "5" },
    { "17", empl17, 24, 4, validAge + "24"
+ validXpYears + "4" },
    { "AgeError_4", emplfault1, 70, 47,
ageBetween18And70 },
    { "YearsOfXpError_5", emplfault2, 18, 0,
youngerThan18ForHiring }

};

// Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)
int i = 0;
bool failed = false;

// Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου
for (i = 0; i < testcases.GetLength(0); i++)
// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases
{
    try
    {
        Employee TestcaseEmployee = (Employee)testcases[i, 1]; // Ο υπάλληλος του testcase i
        int ExpectedAge = (int)testcases[i, 2]; // Η ηλικία του υπαλλήλου του testcase i που
        περιμένω να επιστρέψει η InfoEmployee()
```


ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
int ExpectedYearsOfExperience = (int)testcases[i, 3];    // Τα χρόνια υπηρεσίας του υπαλλήλου του
testcase i που περιμένω να επιστρέψει η InfoEmployee()

// Δήλωση και αρχικοποίηση ref μεταβλητών μεθόδου
int ActualAge = 0;
int ActualYearsOfExperience = 100;

// Κλήση της InfoEmployee με χρήση ref για να επιστραφούν οι τιμές ActualAge και
ActualYearsOfExperience
// καθώς και η μορφοποίηση της ημερομηνίας FormattedDate
per.InfoEmployee(TestcaseEmployee, ref ActualAge, ref ActualYearsOfExperience);

// Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,
// δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases
Assert.AreEqual(ExpectedAge, ActualAge);
Assert.AreEqual(ExpectedYearsOfExperience, ActualYearsOfExperience);
}
catch (Exception e)
{
    // Απέτυχε το Test Case
    failed = true;
    // Καταγράφουμε το Test Case που απέτυχε
    Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξαίρεση: {2} ",
        (string)testcases[i, 0], (string)testcases[i, 4], e.Message);
    // id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου ελέγχου,
    Μήνυμα του exception
}
}

// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception.
if (failed)
    Assert.Fail();
}
```

6. LiveInAthens Unit Test

```
[TestMethod]
public void TestMethodLiveInAthens()
{
    // Παραδοχές υλοποίησης
    string numAthensEmps = " Υπάλληλοι που κατοικούν στην Αθήνα : ";

    // Δημιουργία ενός αντικειμένου της κλάσης Personnel του HRLib.dll που θέλουμε να τεστάρουμε
    HRLib.Personnel per = new HRLib.Personnel();

    Employee[] empls1 = new Employee[]
    {
        // "Όνοματεπώνυμο", "Σταθερό Τηλέφωνο", "Κινητό Τηλέφωνο", "Ημερομηνία Γέννησης",
        "Ημερομηνία Πρόσληψης"
        new Employee("George Theocharis", "2202625345", "6971345456", new DateTime(2001, 08,
23), new DateTime(2023, 07, 03)),
        new Employee("Vasilis Athanasiou", "2302324901", "6981314145", new DateTime(2001, 03, 19),
new DateTime(2024, 01, 02)),
        new Employee("Omar Alhaz", "2401234681", "6945678789", new DateTime(2001, 03, 24),
new DateTime(2023, 12, 20))
    };
    Employee[] empls2 = new Employee[]
    {
        // "Όνοματεπώνυμο", "Σταθερό Τηλέφωνο", "Κινητό Τηλέφωνο", "Ημερομηνία Γέννησης",
        "Ημερομηνία Πρόσληψης"
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

    new Employee("Tom Hanks", "2102625345", "6971345456", new DateTime(1959, 07, 09),
new DateTime(1989, 06, 09)),
    new Employee("Leonardo DiCaprio", "2102324901", "6981314145", new DateTime(1974, 11,
11), new DateTime(1995, 03, 15)),
    new Employee("Meryl Streep", "2801234681", "6945678789", new DateTime(1955, 06, 22),
new DateTime(1975, 05, 05)),
    new Employee("Robert Downey", "2102625345", "6971345456", new DateTime(1965, 04, 04),
new DateTime(1984, 08, 23)),
    new Employee("Johnny Depp", "2102324901", "6981314145", new DateTime(1963, 06, 09),
new DateTime(1984, 09, 10)),
    new Employee("Angelina Jolie", "2801234681", "6945678789", new DateTime(1975, 06, 04),
new DateTime(1994, 10, 12)),
    new Employee("Chris Hemsworth", "2102324901", "6981314145", new DateTime(1983, 08,
11), new DateTime(2002, 03, 18)),
};
Employee[] empl3 = new Employee[]
{
    // "Όνοματεπώνυμο", "Σταθερό Τηλέφωνο", "Κινητό Τηλέφωνο", "Ημερομηνία Γέννησης",
    "Ημερομηνία Πρόσληψης"
    new Employee("Brad Pitt", "2702625345", "6971345456", new DateTime(1963, 12, 18),
new DateTime(1987, 05, 20)),
    new Employee("Jennifer Aniston", "2602324901", "6981314145", new DateTime(1969, 02, 11),
new DateTime(1992, 07, 15)),
    new Employee("Tom Cruise", "2801234681", "6945678789", new DateTime(1962, 07, 03),
new DateTime(1981, 10, 10)),
    new Employee("George Clooney", "2102625345", "6971345456", new DateTime(1961, 05, 06),
new DateTime(1982, 09, 22)),
    new Employee("Julia Roberts", "2102324901", "6981314145", new DateTime(1967, 10, 28),
new DateTime(1988, 07, 15)),
    new Employee("Denzel Washington", "2801234681", "6945678789", new DateTime(1954, 12,
28), new DateTime(1978, 06, 12)),
    new Employee("Scarlett Johansson", "2102625345", "6971345456", new DateTime(1984, 11, 22),
new DateTime(2003, 05, 10)),
    new Employee("Robert Pattinson", "2801234681", "6945678789", new DateTime(1986, 05, 13),
new DateTime(2004, 11, 07))
};
Employee[] empl4 = new Employee[]
{
    // "Όνοματεπώνυμο", "Σταθερό Τηλέφωνο", "Κινητό Τηλέφωνο", "Ημερομηνία Γέννησης",
    "Ημερομηνία Πρόσληψης"
    new Employee("George Theoxaris", "2102625345", "6971345456", new DateTime(2001,
08, 23), new DateTime(2023, 07, 03)),
    new Employee("Vasilis Athanasiou", "2102324901", "6981314145", new DateTime(2001,
03, 19), new DateTime(2024, 01, 02)),
    new Employee("Omar Alhaz", "2801234681", "6945678789", new DateTime(2001, 03,
24), new DateTime(2023, 12, 20)),
    new Employee("Christos Troussas", "2131414567", "6941296963", new DateTime(1994,
11, 15), new DateTime(2024, 01, 01)),
    new Employee("Akrivi Krouska", "2215678901", "6901234345", new DateTime(1998, 05,
19), new DateTime(2023, 12, 20)),
    new Employee("Ioannis Vogiatzis", "2801234681", "6945678789", new DateTime(2001, 03,
24), new DateTime(2023, 12, 20)),
    new Employee("George Meletiou", "2461345679", "6931345450", new DateTime(1980,
06, 04), new DateTime(2023, 11, 03)),
    new Employee("Panagiotis Giannakopoulos", "2151234510", "6901234342", new
DateTime(1978, 06, 10), new DateTime(2024, 02, 20)),
    new Employee("Stavros Fatouros", "2612828456", "6951616987", new DateTime(1990,
01, 24), new DateTime(2023, 03, 11))
};
Employee[] empl5 = new Employee[]
{

```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
//      "Όνοματεπώνυμο",      "Σταθερό Τηλέφωνο",  "Κινητό Τηλέφωνο",      "Ημερομηνία Γέννησης",  
"Ημερομηνία Πρόσληψης"  
new Employee("Bartlomiej Dragowski", "2145678901", "6905678901", new DateTime(1997,  
08, 19), new DateTime(2024, 01, 19)),  
new Employee("Tin Jedvaj", "2156789012", "6935678901", new DateTime(1995, 11,  
28), new DateTime(2023, 07, 14)),  
new Employee("Filip Mladenovic", "2167890123", "6995678901", new DateTime(1991, 08,  
15), new DateTime(2023, 07, 01)),  
new Employee("Giannis Kotsiras", "2178901234", "6945678901", new DateTime(1992, 12,  
16), new DateTime(2021, 07, 01)),  
new Employee("William Arao", "2189012345", "6955678901", new DateTime(1992, 03,  
12), new DateTime(2023, 08, 17)),  
new Employee("Carlos Zeca", "2190123456", "6975678901", new DateTime(1988, 08,  
21), new DateTime(2023, 07, 01)),  
new Employee("Bernard Duarte", "2101234567", "6985678901", new DateTime(1992,  
09, 08), new DateTime(2022, 08, 21)),  
new Employee("Anastasios Bakasetas", "2112345678", "6935678901", new DateTime(1993,  
06, 28), new DateTime(2024, 01, 22)),  
new Employee("Adam Cerin", "2123456789", "6945678901", new DateTime(1999, 07,  
16), new DateTime(2022, 07, 02)),  
new Employee("Sebastian Palacios", "2134567890", "6975678901", new DateTime(1992,  
01, 20), new DateTime(2021, 08, 31)),  
new Employee("Fotis Ioannidis", "2145678901", "6985678901", new DateTime(2000, 01,  
10), new DateTime(2020, 08, 10))  
};  
Employee[] empls6 = new Employee[]  
{  
    //      "Όνοματεπώνυμο",      "Σταθερό Τηλέφωνο",  "Κινητό Τηλέφωνο",      "Ημερομηνία  
Γέννησης",      "Ημερομηνία Πρόσληψης"  
    new Employee("George Theoxaris", "2102625345", "6971345456", new DateTime(2001,  
08, 23), new DateTime(2023, 07, 03)),  
    new Employee("Vasilis Athanasiou", "2402324901", "6981314145", new DateTime(2001,  
03, 19), new DateTime(2024, 01, 02)),  
    new Employee("Omar Alhaz", "2501234681", "6945678789", new DateTime(2001, 03,  
24), new DateTime(2023, 12, 20)),  
    new Employee("Dimitra Papadopoulou", "2604567890", "6934567890", new DateTime(2000,  
01, 15), new DateTime(2023, 06, 20)),  
    new Employee("Thanasis Papanikolaou", "2705678901", "6945678901", new DateTime(1999,  
11, 28), new DateTime(2019, 03, 05)),  
    new Employee("Eirini Nikolaou", "2806789012", "6956789012", new DateTime(2001, 09,  
8), new DateTime(2023, 08, 12)),  
    new Employee("Panagiotis Georgopoulos", "2107890123", "6907890123", new DateTime(2000,  
07, 22), new DateTime(2019, 01, 01))  
};  
Employee[] empls7 = new Employee[]  
{  
    //      "Όνοματεπώνυμο",      "Σταθερό Τηλέφωνο",  "Κινητό Τηλέφωνο",      "Ημερομηνία  
Γέννησης",      "Ημερομηνία Πρόσληψης"  
    new Employee("Despina Vandi", "2201234567", "6931234567", new DateTime(1964,  
07, 22), new DateTime(1998, 10, 01)),  
    new Employee("Sakis Rouvas", "2102345678", "6942345678", new DateTime(1972, 01,  
05), new DateTime(1990, 05, 15)),  
    new Employee("Kostas Martakis", "2703456789", "6953456789", new DateTime(1974,  
09, 14), new DateTime(2000, 12, 30)),  
    new Employee("Elena Paparizou", "2804567890", "6944567890", new DateTime(1982,  
01, 31), new DateTime(2010, 02, 20)),  
    new Employee("Giorgos Liagas", "2605678901", "6975678901", new DateTime(1974, 10,  
26), new DateTime(2001, 07, 10))  
};  
  
// Δημιουργία Περιπτώσεων Ελέγχου (Test Cases)  
object[,] testcases =
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
{
// { id,          "Λίστα Υπαλλήλων",  εκτιμώμενη τιμή επιστροφής,      "Παραδοχή υλοποίησης"}
//                                     της LiveInAthens()
{ "1",          empls1,          0,          numAthensEmps + "0/3" },
{ "2",          empls2,          5,          numAthensEmps + "5/7" },
{ "3",          empls3,          3,          numAthensEmps + "3/8" },
{ "4",          empls4,          4,          numAthensEmps + "3/9" },
{ "5",          empls5,          11,         numAthensEmps + "11/11" },
{ "6",          empls6,          2,          numAthensEmps + "3/7" },
{ "7",          empls7,          1,          numAthensEmps + "1/5" }

};

// Αρχικοποίηση δείκτη περιπτώσεων ελέγχου (Test Cases)
int i = 0;
bool failed = false;

// Προσπέλαση και εκτέλεση περιπτώσεων ελέγχου
for (i = 0; i < testcases.GetLength(0); i++)
// Για κάθε περίπτωση ελέγχου (Test Case), δηλαδή για κάθε γραμμή i του πίνακα testcases
{
    try
    {
        Employee[] TestcaseEmps = (Employee[])testcases[i, 1];          // Η λίστα των υπαλλήλων του
testcase i
        int ExpectedAthensHabitants = (int)testcases[i, 2];          // Οι κάτοικοι Αθήνας του testcase i που
περιμένω να επιστρέψει η LiveInAthens()
        int ActualAthensHabitants = per.LiveinAthens(TestcaseEmps);    // Οι κάτοικοι Αθήνας του testcase
i που επιστρέφει η LiveInAthens()

        // Καλούμε την Assert.AreEqual δίνοντας ως παραμέτρους τα στοιχεία της περίπτωσης ελέγχου,
        // δηλαδή τα αντίστοιχα στοιχεία της γραμμής i του πίνακα testcases
        Assert.AreEqual(ExpectedAthensHabitants, ActualAthensHabitants);
    }
    catch (Exception e)
    {
        // Απέτυχε το Test Case
        failed = true;
        // Καταγράφουμε το Test Case που απέτυχε
        Console.WriteLine("Αποτυχημένο Test Case: {0} \n \t Παραδοχή: {1} \n \t Εξαίρεση: {2} ",
            (string)testcases[i, 0], (string)testcases[i, 3], e.Message);
        // id, Παραδοχή υλοποίησης που παραβιάζεται ή μήνυμα έγκυρου
ελέγχου, Μήνυμα του exception
    }
}

// Στην περίπτωση που κάποιο Test Case απέτυχε, πέταξε exception.
if (failed)
    Assert.Fail();
}
```

Δ. Αναφορές Ελέγχου

1. Σφάλμα Ονοματεπωνύμου Υπαλλήλου

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

1.1. Αναγνωριστικό Σφάλματος

NameError_1

1.2. Όνομα μεθόδους ελέγχου που απέτυχε

TestMethodValidName

1.3. Περιγραφή ελέγχου που πραγματοποιήθηκε

Ο έλεγχος πραγματοποιείται προκειμένου να διαπιστωθεί αν το κειμενικό περιεχόμενο του δεδομένου εισόδου αντιστοιχεί σε κάποιο ονοματεπώνυμο υπαλλήλου. Διεξοδικά, αν το κειμενικό περιεχόμενο τηρεί όλες τις παραδοχές υλοποίησης της μεθόδου ValidName() που αναφέρθηκαν [εδώ](#), τότε αντιστοιχεί σε έγκυρο ονοματεπώνυμο υπαλλήλου. Διαφορετικά, αν παραβιάζει τουλάχιστον μία παραδοχή, τότε δεν μπορεί να αντιστοιχεί σε έγκυρο ονοματεπώνυμο υπαλλήλου.

1.4. Αναφορά Σφάλματος που παρουσιάστηκε

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	Name	Flag	Hint
NameError_1	Vasileios Evangelos Athanasiou	true	firstnameAndSurname

Hint	Message
firstnameAndSurname	[1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά

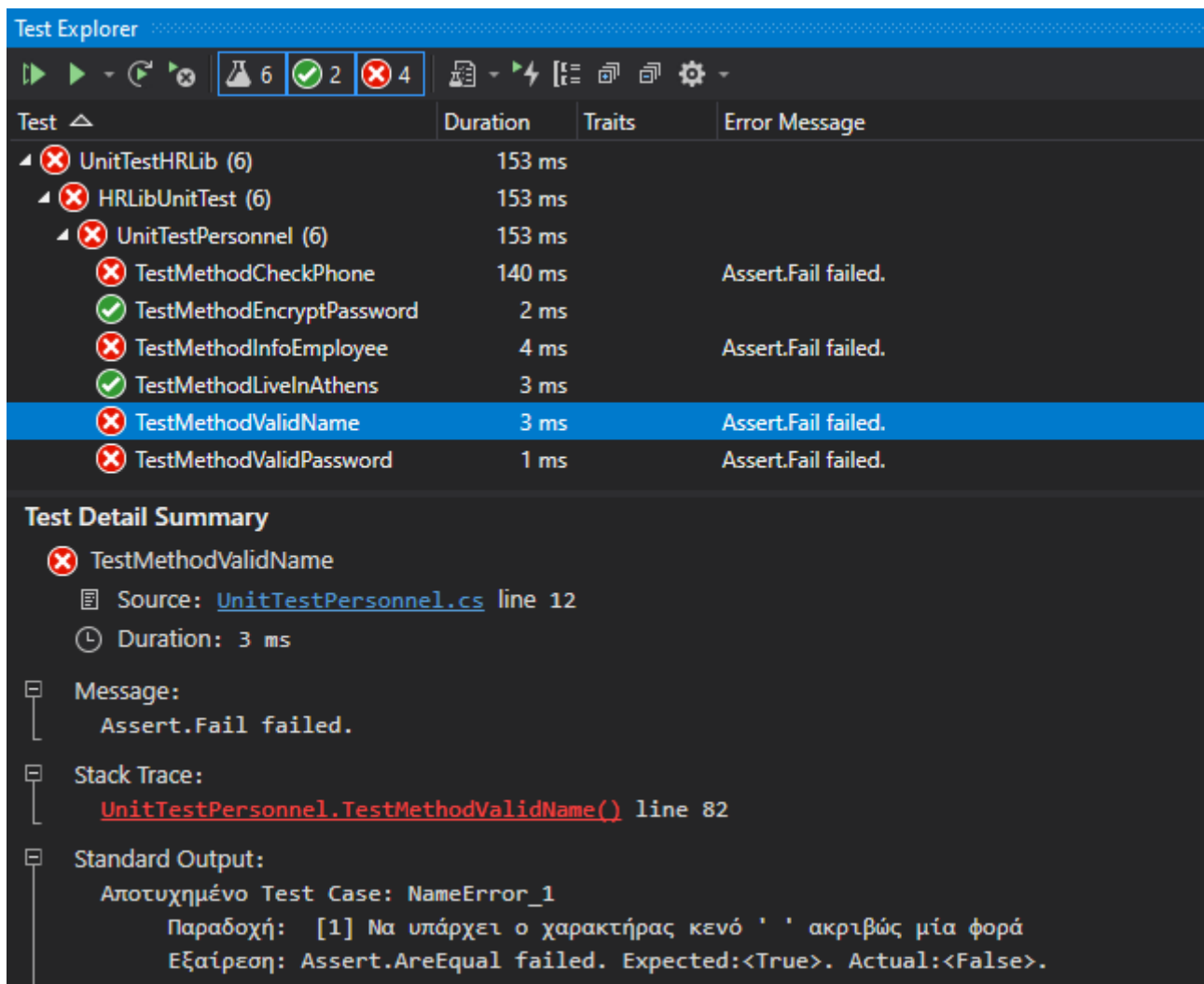
Σύμφωνα με τις παραδοχές υλοποίησης της μεθόδου ValidName() που αναφέρθηκαν [εδώ](#), το δεδομένο εισόδου με #id NameError_1, παραβιάζει την παραδοχή:

[1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά

Το ονοματεπώνυμο του test case με #id NameError_1 περιέχει 2 χαρακτήρες κενό ' ', πράγμα που οδηγεί σε σφάλμα. Ουσιαστικά, το λογισμικό αποτρέπει την εισαγωγή πιθανού 2^{ου} ονόματος ενός υπαλλήλου με αποτέλεσμα να υπάρχει ανακρίβεια όσον αφορά την αξιοπιστία των στοιχείων του.

1.5. Στιγμιότυπο οθόνης του Test Explorer

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ



Test	Duration	Traits	Error Message
✖ UnitTestHRLib (6)	153 ms		
✖ HRLibUnitTest (6)	153 ms		
✖ UnitTestPersonnel (6)	153 ms		
✖ TestMethodCheckPhone	140 ms		Assert.Fail failed.
✔ TestMethodEncryptPassword	2 ms		
✖ TestMethodInfoEmployee	4 ms		Assert.Fail failed.
✔ TestMethodLiveInAthens	3 ms		
✖ TestMethodValidName	3 ms		Assert.Fail failed.
✖ TestMethodValidPassword	1 ms		Assert.Fail failed.

Test Detail Summary

✖ TestMethodValidName

Source: [UnitTestPersonnel.cs](#) line 12

Duration: 3 ms

Message:
Assert.Fail failed.

Stack Trace:
[UnitTestPersonnel.TestMethodValidName\(\)](#) line 82

Standard Output:
Αποτυχημένο Test Case: NameError_1
Παραδοχή: [1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά
Εξαίρεση: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

2. Σφάλμα Κωδικού Πρόσβασης

2.1. Αναγνωριστικό Σφάλματος

PasswordError_2

2.2. Όνομα μεθόδους ελέγχου που απέτυχε

TestMethodValidPassword

2.3. Περιγραφή ελέγχου που πραγματοποιήθηκε

Ο έλεγχος πραγματοποιείται προκειμένου να διαπιστωθεί αν το κειμενικό περιεχόμενο του δεδομένου εισόδου αντιστοιχεί σε κάποιο αποδεκτό κωδικό πρόσβασης. Διεξοδικά, αν το κειμενικό περιεχόμενο τηρεί όλες τις παραδοχές υλοποίησης της μεθόδου ValidPassword() που αναφέρθηκαν

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

[εδώ](#), τότε αντιστοιχεί σε έγκυρο κωδικό πρόσβασης. Διαφορετικά, αν παραβιάζει τουλάχιστον μία παραδοχή, τότε δεν μπορεί να αντιστοιχεί σε αποδεκτό κωδικό πρόσβασης.

2.4. Αναφορά Σφάλματος που παρουσιάστηκε

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα	
#id	Password	Flag	Hint
PasswordError_2	Pao131908~123	true	oneSymbol

Hint	Message
oneSymbol	[3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο

Σύμφωνα με τις παραδοχές υλοποίησης της μεθόδου ValidPassword() που αναφέρθηκαν [εδώ](#), το δεδομένο εισόδου με #id PasswordError_2, παραβιάζει την παραδοχή:

[3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο

Ο κωδικός πρόσβασης του test case με #id PasswordError_2 περιέχει το ειδικό σύμβολο ~, πράγμα που οδηγεί σε σφάλμα. Ουσιαστικά, το λογισμικό δέχεται ως ειδικά σύμβολα μόνο τα σύμβολα !@#\$%^&*()_+~=[\]{}|;:~\".,.<>/? που ανήκουν στο σύνολο αυτό. Το σύμβολο ~ δεν ανήκει στο σύνολο αυτό και με αυτή την παραδοχή, το λογισμικό περιορίζει το πλήθος των αποδεκτών ειδικών συμβόλων που να ικανοποιούν την παραδοχή [3.4] που παραβιάστηκε στα προαναφερόμενα.

2.5. Στιγμιότυπο οθόνης του Test Explorer

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Test	Duration	Traits	Error Message
✖ UnitTestHRLib (6)	153 ms		
✖ HRLibUnitTest (6)	153 ms		
✖ UnitTestPersonnel (6)	153 ms		
✖ TestMethodCheckPhone	140 ms		Assert.Fail failed.
✔ TestMethodEncryptPassword	2 ms		
✖ TestMethodInfoEmployee	4 ms		Assert.Fail failed.
✔ TestMethodLiveInAthens	3 ms		
✖ TestMethodValidName	3 ms		Assert.Fail failed.
✖ TestMethodValidPassword	1 ms		Assert.Fail failed.

Test Detail Summary

✖ TestMethodValidPassword

Source: [UnitTestPersonnel.cs](#) line 86

Duration: 1 ms

Message:
Assert.Fail failed.

Stack Trace:
[UnitTestPersonnel.TestMethodValidPassword\(\)](#) line 157

Standard Output:
Αποτυχημένο Test Case: PasswordError_2
Παραδοχή: [3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο
Εξαίρεση: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

3. Σφάλμα Τηλεφώνου

3.1. Αναγνωριστικό Σφάλματος

PhoneError_3

3.2. Όνομα μεθόδους ελέγχου που απέτυχε

TestMethodCheckPhone

3.3. Περιγραφή ελέγχου που πραγματοποιήθηκε

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Ο έλεγχος πραγματοποιείται προκειμένου να διαπιστωθεί αν το κειμενικό περιεχόμενο του δεδομένου εισόδου αντιστοιχεί σε κάποιο τηλέφωνο υπαλλήλου (είτε σταθερό είτε κινητό). Διεξοδικά, αν το κειμενικό περιεχόμενο τηρεί όλες τις παραδοχές υλοποίησης της μεθόδου CheckPhone() που αναφέρθηκαν [εδώ](#), τότε αντιστοιχεί σε έγκυρο τηλέφωνο υπαλλήλου. Διαφορετικά, αν παραβιάζει τουλάχιστον μία παραδοχή, τότε δεν μπορεί να αντιστοιχεί σε έγκυρο τηλέφωνο υπαλλήλου.

3.4. Αναφορά Σφάλματος που παρουσιάστηκε

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα		
#id	Phone	Type Phone	Info Phone	Hint
PhoneError_3	210 28 12 967	0	Metropolitan Area of Athens - Piraeus	onlyDigits

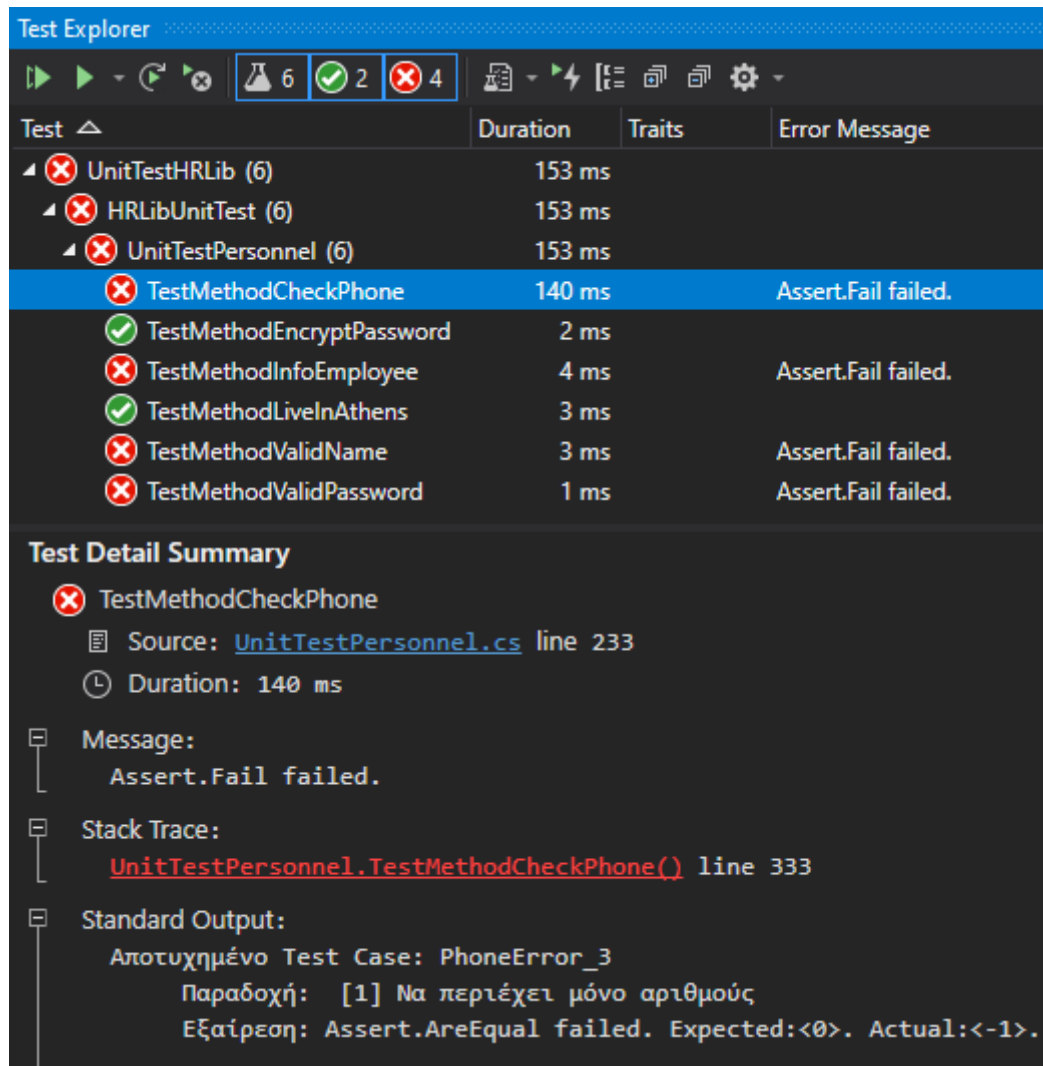
Hint	Message
onlyDigits	[1] Να περιέχει μόνο αριθμούς

Σύμφωνα με τις παραδοχές υλοποίησης της μεθόδου CheckPhone() που αναφέρθηκαν [εδώ](#), το δεδομένο εισόδου με #id PhoneError_3, παραβιάζει την παραδοχή:

[1] Να περιέχει μόνο αριθμούς

Το τηλέφωνο του test case με #id PhoneError_3 δεν περιέχει 10 χαρακτήρες, καθώς υπάρχουν κενά ‘ ‘ ανάμεσα στα ψηφία, πράγμα που οδηγεί σε σφάλμα. Ουσιαστικά, το λογισμικό αποτρέπει την εισαγωγή οποιουδήποτε άλλου χαρακτήρα στο τηλέφωνο πέρα από ψηφία, παρόλο που το τηλέφωνο 210 28 12 967 είναι ένα αποδεκτό σταθερό τηλέφωνο στην ζώνη της Μητροπολιτικής περιοχής της Αθήνας – Πειραιά. Αυτό αποτελεί ένα πρόβλημα για την αξιοπιστία του λογισμικού, καθώς, το διαφορετικό συντακτικό του ορθολογικά σωστού στοιχείου απορρίπτει την εξακρίβωση του.

3.5. Στιγμιότυπο οθόνης του Test Explorer



4. Σφάλμα Ηλικίας Υπαλλήλου

4.1. Αναγνωριστικό Σφάλματος

AgeError_4

4.2. Όνομα μεθόδους ελέγχου που απέτυχε

TestMethodInfoEmployee

4.3. Περιγραφή ελέγχου που πραγματοποιήθηκε

Ο έλεγχος πραγματοποιείται προκειμένου να υπολογιστεί η ηλικία και τα χρόνια προϋπηρεσίας ενός υπαλλήλου. Διεξοδικά, αν οι ημερομηνίες τηρούν όλες τις παραδοχές υλοποίησης της μεθόδου InfoEmployee() που αναφέρθηκαν [εδώ](#), τότε αντιστοιχούν σε έγκυρες ημερομηνίες γέννησης και πρόσληψης αντίστοιχα. Διαφορετικά, αν παραβιάζεται τουλάχιστον μία παραδοχή από μία τουλάχιστον ημερομηνία, τότε δεν υπολογίζεται η ηλικία και τα χρόνια προϋπηρεσίας του υπαλλήλου και συνεπώς οι ημερομηνίες του θεωρούνται άκυρες για τις απαιτήσεις του λογισμικού.

4.4. Αναφορά Σφάλματος που παρουσιάστηκε

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
emplfault1	Aggelos Skandalis	2501010102	6988558589	1953-05-30	1977-02-03

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα		
#id	Employee	Age	Years Of Experience	Hint
AgeError_4	emplFault1	70	47	ageBetween18And70

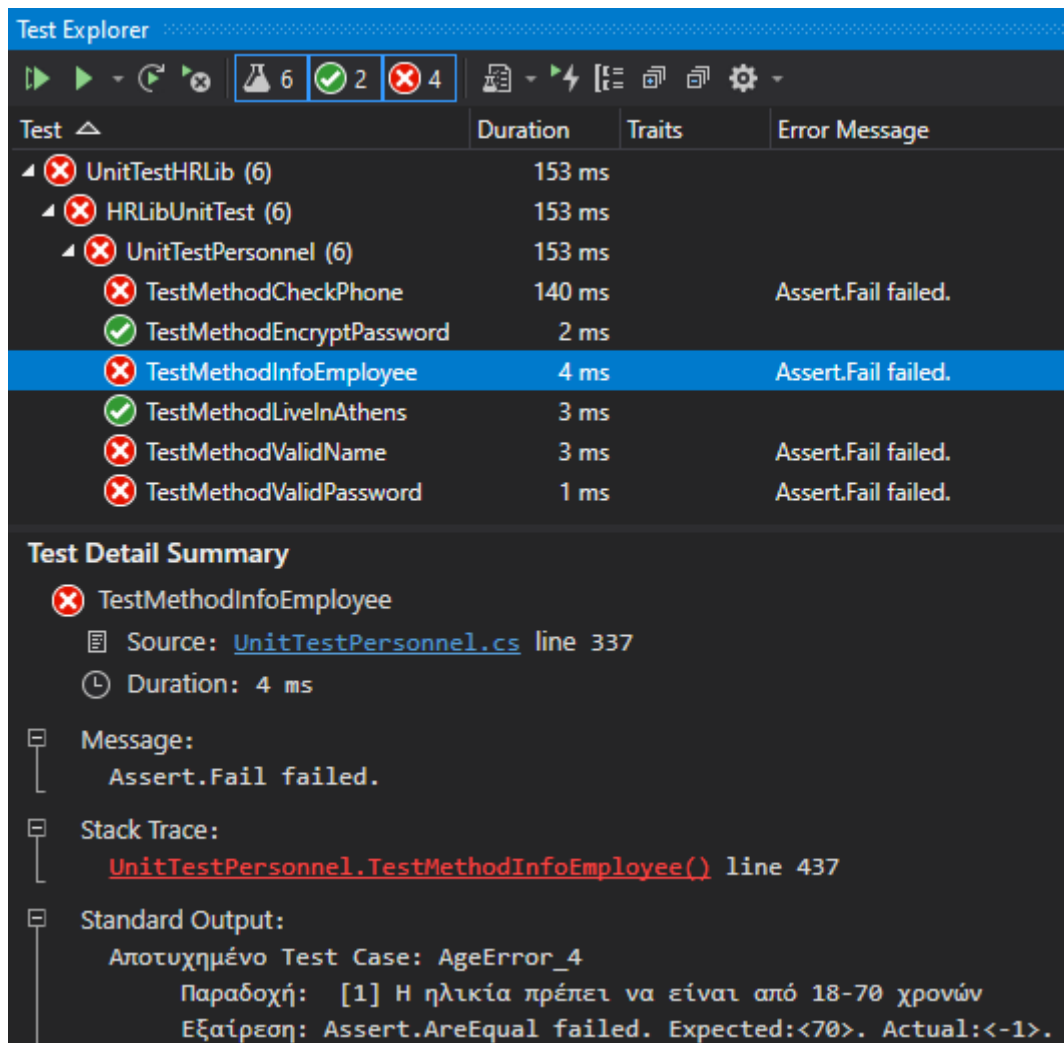
Hint	Message
ageBetween18And70	[1] Η ηλικία πρέπει να είναι από 18-70 χρονών

Σύμφωνα με τις παραδοχές υλοποίησης της μεθόδου InfoEmployee() που αναφέρθηκαν [εδώ](#), το δεδομένο εισόδου με #id AgeError_4, παραβιάζει την παραδοχή:

[1] Η ηλικία πρέπει να είναι από 18-70 χρονών

Η ημερομηνία γέννησης του test case με #id AgeError_4 αντιστοιχεί σε υπάλληλο που έχει ξεπεράσει τα 70 έτη και συνεπώς δεν τηρεί τις απαιτήσεις του λογισμικού για να διατηρηθεί στα δεδομένα της εταιρίας.

4.5. Στιγμιότυπο οθόνης του Test Explorer



5. Σφάλμα στα Χρόνια Προϋπηρεσίας ενός Υπαλλήλου

5.1. Αναγνωριστικό Σφάλματος

YearsOfXpError_5

5.2. Όνομα μεθόδους ελέγχου που απέτυχε

TestMethodInfoEmployee

5.3. Περιγραφή ελέγχου που πραγματοποιήθηκε

Ο έλεγχος πραγματοποιείται προκειμένου να υπολογιστεί η ηλικία και τα χρόνια προϋπηρεσίας ενός υπαλλήλου. Διεξοδικά, αν οι ημερομηνίες τηρούν όλες τις παραδοχές υλοποίησης της μεθόδου InfoEmployee() που αναφέρθηκαν [εδώ](#), τότε αντιστοιχούν σε έγκυρες ημερομηνίες γέννησης και πρόσληψης αντίστοιχα. Διαφορετικά, αν παραβιάζεται τουλάχιστον μία παραδοχή από μία τουλάχιστον ημερομηνία, τότε δεν υπολογίζεται η ηλικία και τα χρόνια προϋπηρεσίας του υπαλλήλου και συνεπώς οι ημερομηνίες του θεωρούνται άκυρες για τις απαιτήσεις του λογισμικού.

5.4. Αναφορά Σφάλματος που παρουσιάστηκε

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
emplfault2	Dimitris Siametis	2104588989	6999999999	2006-01-02	2024-01-01

Περ. Ελ.	Δοκιμαστικά Δεδομένα Εισόδου	Αναμενόμενα Αποτελέσματα		
#id	Employee	Age	Years Of Experience	Hint
YearsOfXpError_5	emplFault2	18	0	youngerThan18ForHiring

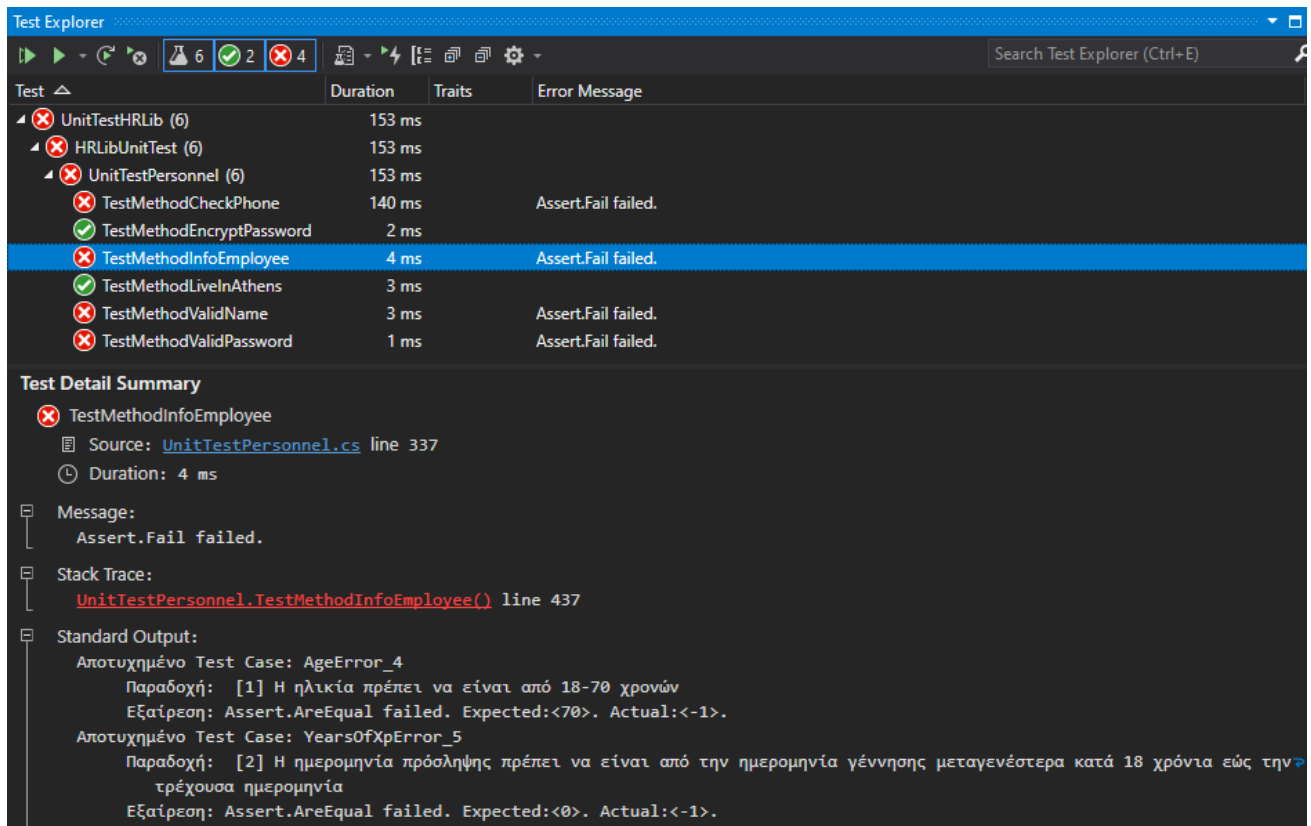
Hint	Message
youngerThan18ForHiring	[2] Η ημερομηνία πρόσληψης πρέπει να είναι από την ημερομηνία γέννησης μεταγενέστερα κατά 18 χρόνια έως την τρέχουσα ημερομηνία

Σύμφωνα με τις παραδοχές υλοποίησης της μεθόδου InfoEmployee() που αναφέρθηκαν [εδώ](#), το δεδομένο εισόδου με #id YearsOfXpError_5, παραβιάζει την παραδοχή:

[2] Η ημερομηνία πρόσληψης πρέπει να είναι από την ημερομηνία γέννησης μεταγενέστερα κατά 18 χρόνια έως την τρέχουσα ημερομηνία

Η ημερομηνία πρόσληψης του test case με #id YearsOfXpError_5 αντιστοιχεί σε υπάλληλο που δεν έχει συμπληρώσει τα 18 έτη και συνεπώς δεν τηρεί τις απαιτήσεις του λογισμικού για να διατηρηθεί στα δεδομένα της εταιρίας.

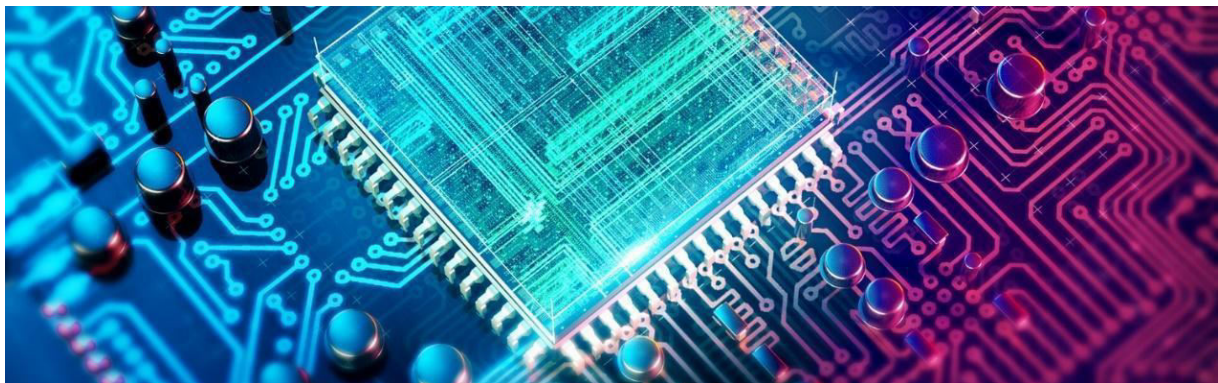
5.5. Στιγμιότυπο οθόνης του Test Explorer



ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ



Σας ευχαριστούμε για την προσοχή σας.



ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

